

Nemesis compression

Nemesis compression is the method created by Sega (/Sega) to reduce the amount of space that graphics data takes up in a Sega Mega Drive (/Sega_Mega_Drive) ROM (/ROM), and is implemented using Yuji Naka (/Yuji_Naka)'s Super Character Maker for MEGA DRIVE (/Super_Character_Maker_for_MEGA_DRIVE).^[1] A decompression algorithm takes the compressed data, unpacks it and transfers it to the VRAM (/index.php?title=VRAM&action=edit&redlink=1). The graphics can then be displayed on the screen. The Nemesis format is specifically designed for Mega Drive graphics, as it relies on the length of the data being divisible by \$20 (hex), which is the exact number of bytes in a single 8x8 tile (/index.php?title=8x8_tile&action=edit&redlink=1). However, Nemesis compression is also used for 16x16 block mappings (/Block_mappings) in Sonic CD, because the data conforms to the \$20 rule.

Nemesis compression is named after Nemesis, the hacker who first created a program to decompress it. The format is used in numerous Mega Drive games, especially Sonic the Hedgehog (/Sonic_the_Hedgehog) games.

Contents

1. Compression theory
 - 1.1. Sega's compressor
2. Format
3. Usage in Mega Drive games
4. Sega Mega Drive games which use Nemesis compression
5. Decompression code
6. References

Compression theory

The Nemesis compression format uses both run-length encoding  and entropy encoding  to compress data. The source data is broken into nybbles which are then grouped into runs of 1 to 8 nybbles in length; there are multiple ways to do this grouping, which is called a *parse*. Given a parse of the file into nybble runs, the file will then be entropy encoded based on the frequency of each nybble run in the file; the code of each nybble run is used to build a dictionary which is used when encoding and decoding the file.

The final Nemesis compressed file is then composed of a 2-byte header, a dictionary associating nybble runs to their respective codes, and a bitstream (the *encoded bitstream*) where each nybble run in the parse is replaced with its corresponding code in the dictionary. The dictionary is present in the compressed file so that it is possible

to decode it; the decoder used in Mega Drive ROMs (/ROM) converts the dictionary into a simple table, using the code read as an index.

There are some restrictions to what forms of entropy encoding can be used: first, is limited to 8 bits for each encoded nybble run; this happens because the dictionary leaves only 8 bits for each code. Second, the set of all codes in the dictionary must form a prefix-free code [\[1\]](#); this makes the encoded bitstream unambiguous in its decoding, but it reduces the number of codes that can be used when entropy-encoding the file. Third, a nibble run can be directly inlined using a special 6-bit code prefix that was specified in the format; because the entropy encoding must result in a prefix-free code, neither this inlining bit pattern, nor any of its prefixes, can be used as codes when building the dictionary.

As noted above, there are multiple ways to parse the input file into nybble runs; each such parse will give rise to different dictionaries because the different frequencies of the nybble runs will change the resulting entropy encoding. Given a parse, the package-merge algorithm [\[2\]](#) can be used to find an optimal dictionary; but since it is not possible to know the length of each dictionary entry beforehand, or how many entries are useful to have in the dictionary, the final file size can't be known during the parsing stage. Finding a parse that gives the smallest possible file size is difficult, particularly since the number of possible parses is staggering even for a small file. Nevertheless, more modern techniques, including the aforementioned package-merge algorithm, have generally improved compression ratios compared to what was found within Mega Drive ROMs (/ROM).

The high compression rate offered by the Nemesis compression format made it a very appropriate format for cartridge-based systems, where storage space was at a premium. Although the Nemesis format provided high compression, it also took a fair amount of time for the 68000 (/68000) CPU to decompress. To prevent noticeable stalls while loading new sets of tiles into the VRAM, the Nemesis decompression routine limited the console to decompressing a maximum of 3 tiles worth of pattern data each frame; some variants of the decoding routines can decode 6 or even 12 tiles in a frame (with a correspondingly larger decompression buffer), but they are much more rarely used.

Sega's compressor

The compressor that was used to compress tile data in *Sonic 1* (/Sonic_1), *Sonic 2* (/Sonic_2), *Sonic 3* (/Sonic_3), *Sonic & Knuckles* (/Sonic_%26_Knuckles), and presumably other titles, used Shannon-Fano coding [\[3\]](#) rather than Huffman coding (/Huffman_Coding). In particular, it used Fano's Method to compute prefix-free codes. A selection sort [\[4\]](#) was used to assign the shortest codes to the most common nybble runs. While Sega's compressor has yet to be found and made publicly available, these behaviours can be recreated to produce a compressor that outputs identical data. [clownnemesisis \(/index.php?title=Clownnemesisis&action=edit&redlink=1\)](/index.php?title=Clownnemesisis&action=edit&redlink=1) is one such compressor.

Format

In order to understand Nemesis compression it is vital to know how the Mega Drive stores graphics. Graphics are stored in the form of 8x8 tiles known as patterns. Each pixel in a pattern is represented by a single nybble, that nybble being the palette (/ Palette#Mega_Drive_Palette) index to use for that pixel. One pattern therefore takes up $8 * 8 = 64$ nybbles or 32 bytes.

Nemesis compression works by replacing the most common nybble runs with codes representing that run. A prefix-free code  system is used, meaning that no valid code can be a prefix of a longer valid code. For example, if 10 is a valid 2-bit code, 110 is a valid 3-bit code but 100 isn't (because it starts with 10). The length of each code is selected according to the frequency of the nybble run that code represents - the most commonly occurring runs will be represented using shorter codes.

The Nemesis compressor can operate in two modes: normal mode, in which each row of pixels in a pattern is encoded as-is, and XOR mode, in which each row of pixels in a pattern is encoded in terms of its differences from the previous row. The first word in a Nemesis-compressed file gives the number of patterns in the uncompressed data (i.e. the size of the uncompressed data / \$20). The sign bit of this word being set indicates that the file uses XOR mode instead of normal mode.

Following this word is a section which describes the codes used in the file. The lower nybble of the first byte gives the palette index. The upper nybble of the next byte gives the number of times that palette index is to be copied - 1, its lower nybble is the length of the code in bits, and the byte after that gives the actual code itself. The byte following this can be interpreted in three ways. If it's FF, it marks the end of the section. Otherwise, if its sign bit is set, its lower nybble gives the new palette index and the two bytes following it have the same format as described above. If its sign bit is clear, however, the palette index remains unchanged and the byte contains the copy count and code length with the actual code being stored in the next byte, as described above. The process repeats until the end of the section is encountered, and a code table is built up in RAM to be used in the next stage of the decompression.

The next section of the file contains the actual compressed data, which is stored as a series of bits. The bit series 111111 has a special purpose: it indicates inline data, and the 7 bits following this series have the format XXXYYYY, where YYYY gives the palette index and XXX gives the copy count - 1. All other series represent a code, and the palette index and copy count for that code are fetched from the code table. When an entire row of a pattern (i.e. 8 pixels/4 bytes long) has been decompressed, it is written directly to the destination if in normal mode, or XORed by the previous row and then written to the destination if in XOR mode. The decompression terminates when the number of decompressed patterns matches the pattern count given in the first word of the file.

Usage in Mega Drive games

The Nemesis compression format was widely used in many Mega Drive games, both those produced by Sega,

as well as a variety of third party developers. Additionally, the decompression routine for Nemesis archives embedded in ROMs that use the format is identical in the games in which it was used. It can be concluded from this that the Nemesis compression format was included as one of the standard libraries provided by Sega in their SDK for the console.

Although Nemesis archives were used for all the compressed art in *Sonic 1* (*/Sonic_1*), for *Sonic 2* (*/Sonic_2*), the main block of art used for level tiles in each stage was compressed using the Kosinski compression (*/Kosinski_compression*) format. This was almost certainly done to decrease load times at the start of the level. Kosinski is an LZ77-based compression algorithm, which provides extremely fast decompression. While the Nemesis format was limited to only 3 tiles of decompressed data per frame, the Kosinski decompression routine may be able to decompress 100 tiles in the same time. This could eliminate up to 3 seconds worth of load times from the start of a level. Although Kosinski compression offered a lightning fast decompression rate, the algorithm was not well suited to the large number of small matches that generally occur in most tiles. As a result, Kosinski compression generally produced a much lower compression ratio than the Nemesis format achieves for Mega Drive art tiles. Additionally, while the Nemesis format could compress any data in 2 passes, the number of passes required with the Kosinski format increased with the size of the data. Kosinski could potentially require hundreds or even thousands of passes over the data to achieve its maximum compression ratios. As a result, the Kosinski format provided much slower compression times, which although insignificant today, may have significantly increased the time it would have taken developers to build test ROMs during development.

For *Sonic 3* (*/Sonic_3*) and *Sonic & Knuckles* (*/Sonic_%26_Knuckles*), most of the art was compressed using Kosinski compression. Despite the lower compression ratio, the use of Kosinski allowed the developers to make large changes to the appearance of the level between acts, and even during the level, without any apparent seams, and without causing visible slowdowns. This was a key factor in creating some of the indoor/outdoor and other mid-level transitions used throughout these games, and allowed the developers to create maps which had much more variety within each level. The extensive use of Kosinski compression for art contributed to the larger ROM size for these later games: 2MB for each game, as opposed to 512KB for *Sonic 1*, and 1MB for *Sonic 2*.

```

+0 +1 +2 +3 +4 +5 +6 +7 +8 +9 +A +B +C +D +E +F 0123456789ABCDEF
31B0 82 72 82 95 82 90 82 85-82 92 20 82 82 82 88 82 Super Chr
31C0 82 82 81 82 83 82 94 82-85 82 92 20 82 8C 82 81 acter Ma
31D0 82 8B 82 85 82 92 20 82-86 82 8F 82 92 20 82 8C ker for M
31E0 82 84 82 86 82 80 20 82-63 82 71 82 88 82 75 82 EGA DRIVE
31F0 84 20 82 75 82 85 82 82-82 51 81 44 82 58 82 55 Ver 2. 96
3200 0D 0A 09 81 83 20 82 63-82 89 82 87 82 89 82 94 ...< Digit
3210 82 89 82 9A 82 85 82 92-20 82 72 82 99 82 93 82 izer Syst
3220 94 82 85 82 8D 20 87 56-20 82 6E 82 8E 82 8C 82 em III Only
3230 99 20 81 84 0D 0A 09 81-40 81 40 81 40 82 6F 82 >... Pr
3240 92 82 8F 82 87 82 92 82-81 82 8D 82 8D 82 85 82 ogrammed
3250 84 20 82 81 82 99 20 82-78 81 44 82 8D 82 81 82 By Y. Nak
3260 8B 82 81 0D 0A 24 0D 0A-83 8A 83 6F 81 5B 83 58 a..$.リバース
3270 20 83 60 83 46 83 62 83-4E 20 81 69 82 4F 81 46 チェック (0:
3280 82 C8 82 B5 20 82 50 81-46 8F E3 89 BA 94 BD 93 なし 1:上下反転
3290 5D 20 82 51 81 46 8D B6-89 45 94 BD 93 5D 20 82 2:左右反転 3
32A0 52 81 46 8F E3 89 BA 8D-B6 89 45 94 BD 93 5D 81 :上下左右反転)
32B0 6A 20 24 0D 0A 82 4F 94-D4 82 CC 83 5A 83 8B 82 $.0番のセルを
32C0 F0 83 58 83 79 81 5B 83-58 82 C9 82 B5 82 DC 82 スペースにします
32D0 B7 82 A9 81 48 20 81 69-82 4F 81 46 82 A2 82 A2 か? (0:いい
32E0 82 A6 20 82 50 81 46 82-CD 82 A2 81 6A 20 24 0D え 1:(はい) $.

```

(/Super_Character_Maker)

Nemesis compression was implemented by Yuji Naka (*/Yuji_Naka*)'s IBM PC (*/IBM_PC*) program Super Character Maker (*/Super_Character_Maker*).

Sega Mega Drive games which use Nemesis compression

This list is vastly incomplete; please help expand it.

- *Alien Storm* (/ *Alien_Storm_(Mega_Drive)*)
- *Bare Knuckle 2* (/ *Bare_Knuckle_2*) (beta only)
- *Castle of Illusion* (/ *Castle_of_Illusion*)
- *Chaotix* (/ *Chaotix*) (32X (/32X))
- *Columns* (/ *Columns*)
- *Dr. Robotnik's Mean Bean Machine* (/ *Dr_Robotnik%27s_Mean_Bean_Machine*) (only for some art — the rest is done using a custom compression introduced in *Puyo Puyo* (/ *Puyo_Puyo*))
- *ESWAT: City Under Siege* (/ *ESWAT:_City_Under_Siege*)
- *Fatal Labyrinth* (/ *Fatal_Labyrinth*)
- *Flicky* (/ *Flicky*)
- *Forgotten Worlds* (/ *Forgotten_Worlds_(Mega_Drive)*)
- *Ghostbusters* (/ *Ghostbusters*)
- *Ghouls 'n' Ghosts* (/ *Ghouls_%27n%27_Ghosts*)
- *Golden Axe* (/ *Golden_Axe*)
- *Golden Axe 2* (/ *Golden_Axe_2*)
- *Golden Axe 3* (/ *Golden_Axe_3*)
- *Jewel Master* (/ *Jewel_Master*) (also contains uncompressed graphics)
- *Magical Taruruto-kun* (/ *Magical_Taruruto-kun_(Mega_Drive)*)
- *Mercs* (/ *Mercs*)
- *Moonwalker* (/ *Moonwalker*)
- *MUSHA* (/ *MUSHA*)
- *Phantasy Star II* (/ *Phantasy_Star_II*) (partially — some art use another simpler compression scheme, some is uncompressed)
- *Phantasy Star II Text Adventures* (/ *Phantasy_Star_II_Text_Adventures*)
- *Phantasy Star III: Generations of Doom* (/ *Phantasy_Star_III:_Generations_of_Doom*)
- *Phantasy Star IV: The End of the Millennium* (/ *Phantasy_Star_IV:_The_End_of_the_Millennium*)
- *Psy-O-Blade* (/ *Psy-O-Blade*)
- *Pulseman* (/ *Pulseman*)
- *Quackshot* (/ *Quackshot*) (graphics)
- *Revenge of Shinobi* (/ *Revenge_of_Shinobi*)
- *Ristar* (/ *Ristar*)
- *Sega Mega Anser* (/ *Sega_Mega_Anser*)
- *Sega Mega-CD* (/ *Sega_Mega-CD*) BIOS
- *Shadow Dancer: The Secret of Shinobi* (/ *Shadow_Dancer:_The_Secret_of_Shinobi*)
- *Sonic & Knuckles* (/ *Sonic_%26_Knuckles*)
- *Sonic Compilation* (/ *Sonic_Compilation*) (menu & games)
- *Sonic Crackers* (/ *Sonic_Crackers*)
- *Sonic Eraser* (/ *Sonic_Eraser*)
- *Sonic Jam* (/ *Sonic_Jam*) (Saturn (/ Saturn))
- *Sonic the Hedgehog* (/ *Sonic_the_Hedgehog_(16-bit)*)
- *Sonic the Hedgehog 2* (/ *Sonic_the_Hedgehog_2_(16-bit)*)
- *Sonic the Hedgehog 3* (/ *Sonic_the_Hedgehog_3*)
- *Sonic the Hedgehog CD* (/ *Sonic_the_Hedgehog_CD*) (Mega-CD (/Mega-CD))
- *Streets of Rage* (/ *Streets_of_Rage*)
- *Streets of Rage 3* (/ *Streets_of_Rage_3*)
- *Strider* (/ *Strider*)
- *Super Hang-On* (/ *Super_Hang-On*)
- *Super Monaco GP* (/ *Super_Monaco_GP*)

- *Twin Cobra* (/Twin_Cobra) (also contains uncompressed graphics)
- *World Cup Italia '90* (/World_Cup_Italia_%2790)
- *World of Illusion Starring Mickey Mouse and Donald Duck* (/World_of_Illusion_Starring_Mickey_Mou

Decompression code

An annotated version of the Nemesis decompression code is provided below for reference. The code is taken from *Sonic 3 & Knuckles* [↗](#), but the same or similar code is used in all the games which use the format.

```

;
_____

; Nemesis decompression subroutine, decompresses art directly to VRAM
; Inputs:
; a0 = art address
; a VDP command to write to the destination VRAM address must be
issued
; before calling this routine
;
_____

```

```

; ===== S U B R O U T I N E
=====

```

```

Nem-Decomp:
    movem.l d0-a1/a3-a5,-(sp)
    lea     (Nem_PCD_WriteRowToVDP).l,a3
    lea     (VDP_data_port).l,a4      ; write all rows to
the VDP data port
    bra.s   Nem-Decomp_Main
; End of function Nem-Decomp

```

```

;
_____

; Nemesis decompression subroutine, decompresses art to RAM
; Inputs:
; a0 = art address
; a4 = destination RAM address
;
_____

```

```

; ===== S U B R O U T I N E
=====

```

```

Nem-Decomp-To-RAM:
    movem.l d0-a1/a3-a5,-(sp)
    lea     (Nem_PCD_WriteRowToRAM).l,a3
; End of function Nem-Decomp-To-RAM

```

```

;

```

; Main Nemesis decompression subroutine

;

; ===== S U B R O U T I N E

Nem-Decomp_Main:

 lea (Nem_code_table).w,a1
 move.w (a0)+,d2 ; get number of patterns
 lsl.w #1,d2
 bcc.s + ; branch if the sign bit isn't set
 adda.w #Nem_PCD_WriteRowToVDP_XOR-

Nem_PCD_WriteRowToVDP,a3 ; otherwise the file uses XOR mode
+

 lsl.w #2,d2 ; get number of 8-pixel rows in the
uncompressed data

 movea.w d2,a5 ; and store it in a5 because there
aren't any spare data registers

 moveq #8,d3 ; 8 pixels in a pattern row
 moveq #0,d2
 moveq #0,d4

 bsr.w Nem_Build_Code_Table

 move.b (a0)+,d5 ; get first byte of compressed
data

 asl.w #8,d5 ; shift up by a byte

 move.b (a0)+,d5 ; get second byte of
compressed data

 move.w #\$10,d6 ; set initial shift value

 bsr.s Nem_Process_Compressed_Data

 movem.l (sp)+,d0-a1/a3-a5

 rts

; End of function Nem-Decomp_Main

;

; Part of the Nemesis decompressor, processes the actual compressed
data

;

```
; ===== S U B R O U T I N E
```

```
; PCD is used throughout this subroutine as an initialism for  
Process_Compressed_Data
```

```
Nem_Process_Compressed_Data:
```

```
    move.w  d6,d7  
    subq.w  #8,d7    ; get shift value  
    move.w  d5,d1  
    lsr.w   d7,d1    ; shift so that high bit of the code  
is in bit position 7
```

```
    cmpi.b  %#11111100,d1    ; are the high 6 bits set?  
    bcc.s   Nem_PCD_InlineData    ; if they are, it  
signifies inline data
```

```
    andi.w  #$FF,d1  
    add.w   d1,d1  
    move.b  (a1,d1.w),d0    ; get the length of the code  
in bits
```

```
    ext.w   d0  
    sub.w   d0,d6    ; subtract from shift value so that  
the next code is read next time around
```

```
    cmpi.w  #9,d6    ; does a new byte need to be read?  
    bcc.s   +        ; if not, branch  
    addq.w  #8,d6  
    asl.w   #8,d5  
    move.b  (a0)+,d5    ; read next byte
```

```
+
```

```
    move.b  1(a1,d1.w),d1  
    move.w  d1,d0  
    andi.w  #$F,d1    ; get palette index for pixel  
    andi.w  #$F0,d0
```

```
Nem_PCD_GetRepeatCount:
```

```
    lsr.w   #4,d0    ; get repeat count
```

```
Nem_PCD_WritePixel:
```

```
    lsl.l   #4,d4    ; shift up by a nybble  
    or.b    d1,d4    ; write pixel  
    subq.w  #1,d3    ; has an entire 8-pixel row been  
written?
```

```
    bne.s   Nem_PCD_WritePixel_Loop ; if not, loop  
    jmp     (a3)    ; otherwise, write the row to its  
destination
```

```
;
```

Nem_PCD_NewRow:

```
    moveq    #0,d4    ; reset row
    moveq    #8,d3    ; reset nybble counter
```

Nem_PCD_WritePixel_Loop:

```
    dbf      d0,Nem_PCD_WritePixel
    bra.s    Nem_Process_Compressed_Data
```

;

Nem_PCD_InlineData:

```
    subq.w   #6,d6    ; 6 bits needed to signal inline data
    cmpi.w   #9,d6
    bcc.s    +
    addq.w   #8,d6
    asl.w    #8,d5
    move.b   (a0)+,d5
```

+

subq.w #7,d6 ; and 7 bits needed for the inline
data itself

```
    move.w   d5,d1
    lsr.w    d6,d1    ; shift so that low bit of the code is
in bit position 0
```

```
    move.w   d1,d0
    andi.w   #$F,d1   ; get palette index for pixel
    andi.w   #$70,d0  ; high nybble is repeat count for
pixel
```

```
    cmpi.w   #9,d6
    bcc.s    Nem_PCD_GetRepeatCount
    addq.w   #8,d6
    asl.w    #8,d5
    move.b   (a0)+,d5
    bra.s    Nem_PCD_GetRepeatCount
```

;

Nem_PCD_WriteRowToVDP:

```
    move.l   d4,(a4)  ; write 8-pixel row
    subq.w   #1,a5
    move.w   a5,d4    ; have all the 8-pixel rows been
```

written?

```
    bne.s    Nem_PCD_NewRow ; if not, branch
    rts      ; otherwise the decompression is finished
```

;

Nem_PCD_WriteRowToVDP_XOR:

```
        eor.l    d4,d2    ; XOR the previous row by the current
row
        move.l   d2,(a4)  ; and write the result
        subq.w   #1,a5
        move.w   a5,d4
        bne.s    Nem_PCD_NewRow
        rts
;

```

Nem_PCD_WriteRowToRAM:

```
        move.l   d4,(a4)+
        subq.w   #1,a5
        move.w   a5,d4
        bne.s    Nem_PCD_NewRow
        rts
;

```

Nem_PCD_WriteRowToRAM_XOR:

```
        eor.l    d4,d2
        move.l   d2,(a4)+
        subq.w   #1,a5
        move.w   a5,d4
        bne.s    Nem_PCD_NewRow
        rts
; End of function Nem_Process_Compressed_Data
;

```

```
; Part of the Nemesis decompressor, builds the code table (in RAM)
;

```

```
; ===== S U B R O U T I N E
=====

```

```
; BCT is used throughout this subroutine as an initialism for
Build_Code_Table
Nem_Build_Code_Table:

```

```

        move.b  (a0)+,d0          ; read first byte

Nem_BCT_ChkEnd:
        cmpi.b  #$FF,d0 ; has the end of the code table
description been reached?
        bne.s   Nem_BCT_NewPalIndex ; if not, branch
        rts     ; otherwise, this subroutine's work is done
;



---



Nem_BCT_NewPalIndex:
        move.w  d0,d7

Nem_BCT_Loop:
        move.b  (a0)+,d0          ; read next byte
        cmpi.b  #$80,d0 ; sign bit being set signifies a new
palette index
        bcc.s   Nem_BCT_ChkEnd ; a bmi could have been used
instead of a compare and bcc
        move.b  d0,d1
        andi.w  #$F,d7 ; get palette index
        andi.w  #$70,d1 ; get repeat count for palette index
        or.w    d1,d7 ; combine the two
        andi.w  #$F,d0 ; get the length of the code in bits
        move.b  d0,d1
        lsl.w   #8,d1
        or.w    d1,d7 ; combine with palette index and
repeat count to form code table entry
        moveq   #8,d1
        sub.w   d0,d1 ; is the code 8 bits long?
        bne.s   Nem_BCT_ShortCode ; if not, a bit of
extra processing is needed
        move.b  (a0)+,d0          ; get code
        add.w   d0,d0 ; each code gets a word-sized entry in
the table
        move.w  d7,(a1,d0.w)      ; store the entry for the code
        bra.s   Nem_BCT_Loop ; repeat
;

```

Sega Retro is part of the Backwards Compatible family of sites, which are dedicated to research and preservation of gaming history and culture.

[Donate \(https://www.patreon.com/segaetro\)](https://www.patreon.com/segaetro) | [Privacy Policy \(https://retrocdn.net/Privacy_Policy\)](https://retrocdn.net/Privacy_Policy) | [About Sega](#)

[Retro](#) | [Contact](#) | [The Nemesis decompressor uses prefix-free codes \(no valid code is a](#)

Except where otherwise noted, content on this site is licensed under a [Creative Commons Attribution 4.0](https://creativecommons.org/licenses/by/4.0/)

[International license \(https://creativecommons.org/licenses/by/4.0/\)](https://creativecommons.org/licenses/by/4.0/).

; e.g. if 10 is a valid 2-bit code, 110 is a valid 3-bit code but 100 isn't

; also, when the actual compressed data is processed the high bit of

each code is in bit position 7

; so the code needs to be bit-shifted appropriately over here before being used as a code table index

; additionally, the code needs multiple entries in the table because no masking is done during compressed data processing

; so if 11000 is a valid code then all indices of the form 11000XXX need to have the same entry

Nem_BCT_ShortCode:

```
        move.b  (a0)+,d0      ; get code
        lsl.w   d1,d0        ; shift so that high bit is in bit
position 7
```

```
        add.w   d0,d0        ; get index into code table
```

```
        moveq   #1,d5
```

```
        lsl.w   d1,d5
```

```
        subq.w  #1,d5        ; d5 = 2^d1 - 1
```

Nem_BCT_ShortCode_Loop:

```
        move.w  d7,(a1,d0.w)  ; store entry
```

```
        addq.w  #2,d0        ; increment index
```

```
        dbf     d5,Nem_BCT_ShortCode_Loop    ; repeat for
required number of entries
```

```
        bra.s   Nem_BCT_Loop
```

```
; End of function Nem_Build_Code_Table
```

References

1. <https://forums.sonicretro.org/threads/everything-that-i-know-about-sonic-the-hedgehogs-source-code.40809/page-16#post-1093032> (<https://web.archive.org/web/20250828035902/https://forums.sonicretro.org/threads/everything-that-i-know-about-sonic-the-hedgehogs-source-code.40809/page-16#post-1093032>) (Wayback Machine: 2025-08-28 03:59)

Retrieved from "https://segaretro.org/index.php?

title=Nemesis_compression&oldid=1119665 (https://segaretro.org/index.php?

title=Nemesis_compression&oldid=1119665)"